UNIVERSIDADE CESUMAR - UNICESUMAR

Curso de Análise e Desenvolvimento de Sistemas

Lyvia Vitoria dos Santos Silva 26006938-2

Gabriel Detomini Rabelo 24385097-2

SISPE

Sistema de Apoio ao Planejamento e ao Acompanhamento de Ações de Saúde Mental no Programa Saúde na Escola

2ª entrega: implementação, documentação e apresentação

Documento referente à 2ª etapa da Atividade Extensionista de Prática (AEP) do curso de Análise e Desenvolvimento de Sistemas da Universidade Cesumar (UniCesumar), 2º semestre de 2026, contendo a implementação em linguagem C, os artefatos de Engenharia de Software revisados, os testes, o manual do usuário e as considerações finais.

Maringá - PR

2026

SUMÁRIO

1 INTRODUÇÃO

2 IMPLEMENTAÇÃO EM LINGUAGEM C

3 ENGENHARIA DE SOFTWARE

3.1 Requisitos do sistema (revisados)

3.2 Diagrama de casos de uso

3.3 Descrição dos casos de uso

3.4 Documentação dos testes

4 MANUAL DO USUÁRIO

5 CONSIDERAÇÕES FINAIS

6 APRESENTAÇÃO EM VÍDEO

REFERÊNCIAS

1 INTRODUÇÃO

Este documento apresenta a 2ª entrega da Atividade Extensionista de Prática (AEP) do grupo responsável pelo SISPE: Sistema de Apoio ao Planejamento e ao Acompanhamento de Ações de Saúde Mental no Programa Saúde na Escola. Na 1ª entrega, o grupo contextualizou o Programa Saúde na Escola (PSE), instituído pelo Decreto nº 6.286 (BRASIL, 2007), definiu a questão norteadora, os objetivos, o escopo ético do sistema, sete requisitos funcionais, quatro requisitos não funcionais, os fluxogramas, os pseudocódigos e o planejamento de seis sprints semanais.

Nesta etapa, o planejamento foi transformado em uma aplicação funcional em linguagem C, executada em terminal, que cadastra, lista, pesquisa e acompanha ações coletivas de saúde mental e gera um resumo de participação por situação, por subtema e por escola. O sistema continua respeitando os limites definidos na 1ª entrega: utiliza apenas dados fictícios e coletivos, não registra informações de estudantes, não realiza diagnóstico e não recomenda tratamento.

O documento está organizado da seguinte forma: a seção 2 descreve a implementação em C; a seção 3 reúne os artefatos de Engenharia de Software (requisitos atualizados, diagrama e descrições de casos de uso e testes); a seção 4 traz o manual do usuário; a seção 5, as considerações finais; e a seção 6, a apresentação em vídeo. O código-fonte e a documentação estão no repositório https://github.com/ashmuchluv/SISPE.

2 IMPLEMENTAÇÃO EM LINGUAGEM C

A aplicação foi escrita em C, utilizando apenas as bibliotecas padrão `stdio.h`, `stdlib.h` e `string.h`, e manteve a forma de construção trabalhada na disciplina de Linguagem e Técnicas de Programação: estrutura (`struct`) para os dados, vetor para armazenar os registros, funções com uma responsabilidade cada, estruturas condicionais (`if`/`else` e `switch`) e laços de repetição (`for`, `while` e `do-while`). Os dados permanecem em memória durante a execução, conforme o requisito não funcional 2 da 1ª entrega.

2.1 Organização dos arquivos

Seguindo o requisito não funcional 4 (código modular, com responsabilidades separadas por arquivo), o projeto foi dividido em um arquivo principal, um módulo de ações e um módulo de entrada de dados:

Quadro 1: Arquivos do código-fonte

Arquivo	Responsabilidade
`main.c`	Exibe o menu e controla o laço principal (`do-while` + `switch`), chamando a função de cada opção.
`acoes.h`	Constantes (`MAX_ACOES`, situações), a estrutura `Acao` e os protótipos das funções do módulo de ações.
`acoes.c`	As seis funcionalidades: cadastrar, listar, pesquisar, atualizar situação, registrar participantes e gerar resumo.
`entrada.h`	Protótipos das funções de leitura e validação.
`entrada.c`	Leitura segura do teclado (números, textos e datas) e comparação de textos sem diferenciar maiúsculas.

Fonte: os autores (2026).

2.2 Estrutura de dados

Cada ação coletiva do PSE é representada pela estrutura `Acao`. Nenhum campo identifica estudantes: o único dado de pessoa é o responsável pela ação, um profissional da equipe. O estresse médio é um valor agregado do grupo participante, como definido na delimitação de escopo da 1ª entrega.

```
typedef struct {
    int codigo;
    char escola[TAM_TEXTO];
    char subtema[TAM_TEXTO];
    char dataPrevista[TAM_DATA];    // formato DD/MM/AAAA
    char publicoAlvo[TAM_TEXTO];    // ex.: 6o ano, ensino medio
    char responsavel[TAM_TEXTO];    // profissional que conduz a acao
    int quantidadePrevista;
    int situacao;                   // 1 planejada, 2 realizada, 3 cancelada
    int quantidadeRealizada;        // preenchida somente quando realizada
    float estresseMedio;            // media relatada pelo grupo, de 0 a 10
} Acao;
```

As ações ficam no vetor `Acao acoes[MAX_ACOES]`, declarado na função `main`, com capacidade para 50 registros. A variável `totalAcoes` guarda quantas posições estão ocupadas. A função de cadastro recebe o vetor e o total e devolve o novo total, sem variáveis globais para os dados. A única variável global é o vetor `nomeSituacao`, que guarda os nomes Planejada, Realizada e Cancelada na posição do código de cada situação. Para o resumo, uma segunda estrutura, `Grupo`, guarda os totais de cada subtema ou escola.

2.3 Funções

Quadro 2: Funções do sistema

Função	Opção	O que faz
`cadastrarAcao` e `lerDadosAcao`	1	Verifica o limite e lê os dados; `lerDadosAcao` é o caso de uso "Validar dados da ação" e só grava se todos os campos forem válidos.
`listarAcoes`	2	Mostra todas as ações em tabela.
`pesquisarAcoes`	3	Pesquisa por código, ou por parte do nome da escola ou do subtema.
`atualizarSituacao`	4	Muda a situação; ao marcar Realizada, pede participantes e estresse médio.
`registrarParticipantes`	5	Registra ou corrige os participantes de uma ação realizada.
`gerarResumo`, `agruparAcoes` e `mostrarGrupos`	6	Somam os totais geral, por subtema e por escola e mostram as tabelas.
`localizarAcao` e `buscarPorCodigo`	3, 4, 5	Caso de uso "Localizar ação pelo código": devolvem a posição da ação ou avisam que ela não existe.
`lerInteiro`, `lerReal`, `lerTexto` e `dataValida`	todas	Leem o teclado com `fgets` e recusam letras em campos numéricos, campos vazios, textos longos e datas inexistentes.

Fonte: os autores (2026).

2.4 Validações e tratamento de erros

As validações seguem o pseudocódigo `cadastrarAcao` da 1ª entrega: ao encontrar um erro, o sistema mostra uma mensagem clara, cancela a operação sem salvar nada e volta ao menu. Foram tratadas as situações abaixo:

- **Opção inválida no menu** (número fora de 0 a 6 ou letras): mensagem de erro e o menu é exibido de novo.
- **Código duplicado**: a função `buscarPorCodigo` percorre o vetor antes de aceitar o código.
- **Código inválido**: aceita apenas números inteiros maiores que zero.
- **Campos obrigatórios vazios** ou formados só por espaços, e textos com mais de 49 caracteres.
- **Data inválida**: formato diferente de DD/MM/AAAA ou data inexistente (ex.: 31/02/2026).
- **Quantidades**: prevista de 1 a 5000; efetiva de 1 a 5000 (se ninguém participou, a ação deve ser cancelada); estresse médio de 0 a 10, aceitando vírgula ou ponto.
- **Registro inexistente** na pesquisa, na atualização e no registro de participantes.
- **Limite de armazenamento**: com 50 ações, o cadastro é recusado antes de pedir os dados.

- **Consistência dos dados**: participantes só são registrados em ações realizadas; se uma ação realizada volta a planejada ou é cancelada, os participantes são zerados para não entrarem no resumo; a mesma situação não é gravada duas vezes.

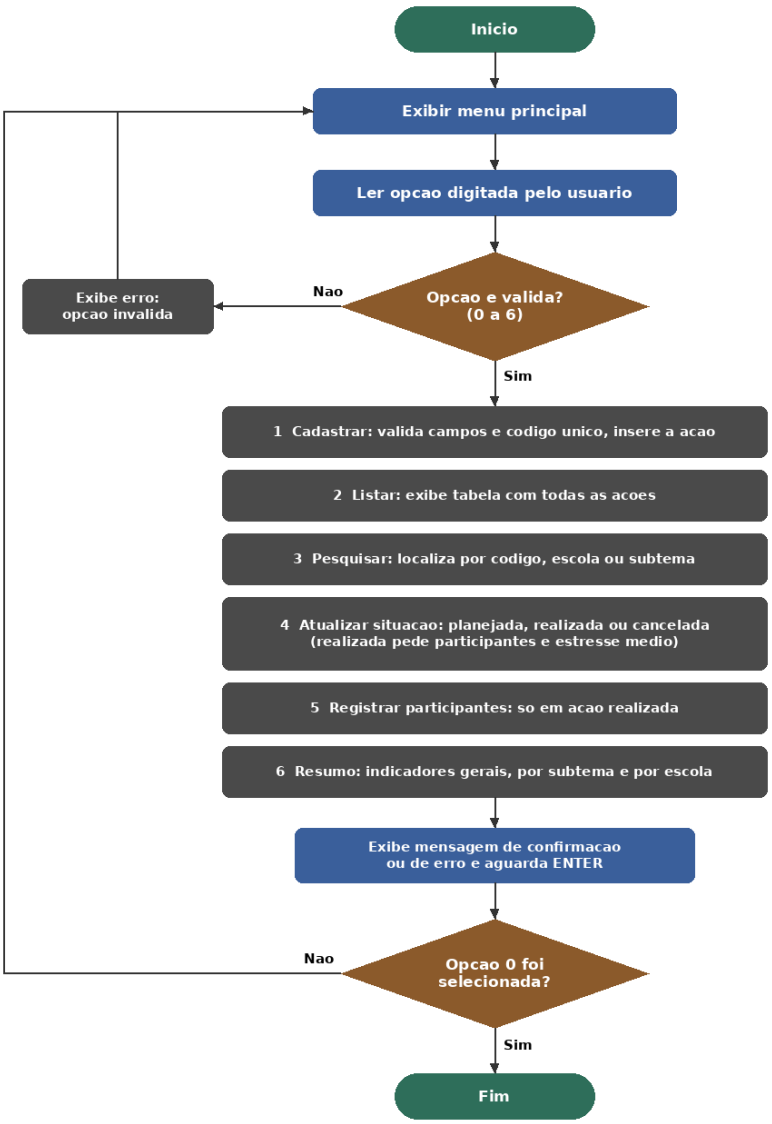
A leitura com `scanf("%d")` entra em laço infinito quando o usuário digita letras. Por isso, toda leitura usa `fgets` para ler a linha inteira e `sscanf` para converter. Quando o valor não é um número, `lerInteiro` devolve -1, valor que nenhum campo do sistema aceita, e a mensagem de erro é exibida.

2.5 Relação com o planejamento da 1ª entrega

O fluxograma geral (Figura 1 da 1ª entrega) foi implementado pelo laço `do-while` da função `main`, que repete o menu até a opção 0, e pelo `switch`, cuja cláusula `default` trata a opção inválida. O pseudocódigo `cadastrarAcao` corresponde à função de mesmo nome, e o pseudocódigo e o fluxograma detalhado de `gerarResumo` correspondem às funções `gerarResumo` e `agruparAcoes`: inicializar contadores, percorrer o vetor, somar por situação, acumular por subtema e por escola e calcular o percentual.

Como foi incluída a opção “Registrar participantes”, o menu passou de 0 a 5 para 0 a 6. O fluxograma geral foi atualizado e é apresentado abaixo.

Figura 1: Fluxograma geral do menu do SISPE (atualizado na 2ª entrega)



Fonte: os autores (2026).

A relação entre cada requisito, a função que o implementa, o caso de uso e os testes está no quadro de rastreabilidade da documentação dos testes.

2.6 Compilação e ambiente de testes

O programa foi compilado com o GCC 11.4.0 no Ubuntu 22.04, com todos os avisos ativados, sem nenhum erro ou aviso:

```
gcc -std=c99 -Wall -Wextra -pedantic main.c acoes.c entrada.c -o sispe
```

Como o código usa apenas a biblioteca padrão, ele também pode ser compilado no Windows com o MinGW ou com o Dev-C++, conforme o manual do usuário (seção 4). O

grupo deve confirmar a compilação no computador do laboratório antes da apresentação, pois os testes registrados neste documento foram executados no ambiente Linux descrito acima.

3 ENGENHARIA DE SOFTWARE

3.1 Requisitos do sistema (revisados)

Os requisitos da 1ª entrega foram revisados durante a implementação. A coluna “Situação” indica se o requisito foi mantido, atualizado ou incluído nesta etapa, e a coluna “Justificativa” explica cada mudança.

Quadro 3: Requisitos funcionais revisados

ID	Requisito funcional	Situação	Justificativa
RF01	Cadastrar ação com código informado pelo usuário, escola, subtema, data prevista, público-alvo, responsável e quantidade prevista. Toda ação nova começa como Planejada.	Atualizado	O pseudocódigo previa “GERAR novoCodigo”. O enunciado da AEP pede o cadastro “informando código” e a 2ª etapa exige impedir códigos duplicados, por isso o código passou a ser digitado e verificado.
RF02	Listar todas as ações cadastradas em tabela organizada.	Mantido	Não se aplica
RF03	Pesquisar ações por código, escola ou subtema.	Atualizado	Escola e subtema podem ser pesquisados por parte do nome, sem diferenciar maiúsculas, para facilitar a localização.
RF04	Atualizar a situação entre planejada, realizada e cancelada. Ao marcar como realizada, registrar a quantidade efetiva de participantes (mínimo 1) e o estresse médio do grupo (0 a 10).	Atualizado	A escala do estresse médio não havia sido definida; adotou-se 0 a 10. Se a ação deixar de ser realizada, os participantes são zerados para manter o resumo consistente.
RF05	Gerar resumo geral com quantidade de ações por situação, total previsto, total de participantes, percentual de participação e estresse médio.	Atualizado	O percentual compara os participantes efetivos com o previsto das ações realizadas, pois ações planejadas ou canceladas ainda não tiveram participação. O total previsto de todas as ações continua exibido.
RF06	Gerar resumo agregado por subtema e por escola.	Mantido	Nomes iguais com maiúsculas diferentes (ex.: Bullying e bullying) formam um único grupo.
RF07	Validar entradas: código duplicado ou inválido, quantidades fora do intervalo, campos vazios, textos longos, datas inválidas e opções inexistentes.	Atualizado	Foram acrescentadas as validações de data, tamanho do texto e entradas não numéricas.
RF08	Registrar ou corrigir os participantes de uma ação já realizada (opção 5 do menu).	Novo	O critério 1 da 2ª etapa avalia “registrar participantes” como funcionalidade própria. Também permite corrigir a contagem depois do evento.
RF09	Informar quando o limite de 50 ações for atingido e recusar novos cadastros.	Novo	Exigência da 2ª etapa: tratar o limite de armazenamento.

Fonte: os autores (2026).

Quadro 4: Requisitos não funcionais revisados

ID	Requisito não funcional	Situação	Observação
RNF01	Funcionar em terminal, sem bibliotecas externas ou banco de dados.	Mantido	Usa somente `stdio.h`, `stdlib.h` e `string.h`.
RNF02	Responder de forma imediata, com os dados em memória durante a execução.	Mantido	Os dados são perdidos ao encerrar; o uso de arquivos fica como melhoria futura.
RNF03	Mensagens compreensíveis para usuário não técnico.	Mantido	Os textos do programa foram escritos sem acentos para aparecerem corretamente em qualquer terminal, inclusive o do Windows.
RNF04	Código modular, com responsabilidades separadas por arquivo.	Mantido	`main.c`, `acoes.c`/`acoes.h` e `entrada.c`/`entrada.h`.
RNF05	Não armazenar dados individuais ou clínicos de estudantes.	Novo	Formaliza como requisito a delimitação de escopo da seção 5 da 1ª entrega.

RNF06	Compilar com um compilador C padrão (C99 ou superior) sem erros nem avisos.	Novo	Atende à exigência de compilar e executar no ambiente definido pelo professor.

Fonte: os autores (2026).

Além dos requisitos, o menu do fluxograma geral passou de 0 a 5 para 0 a 6, por causa do RF08. Os objetivos, o público, o tema (saúde mental no PSE) e os limites éticos da 1ª entrega não foram alterados.

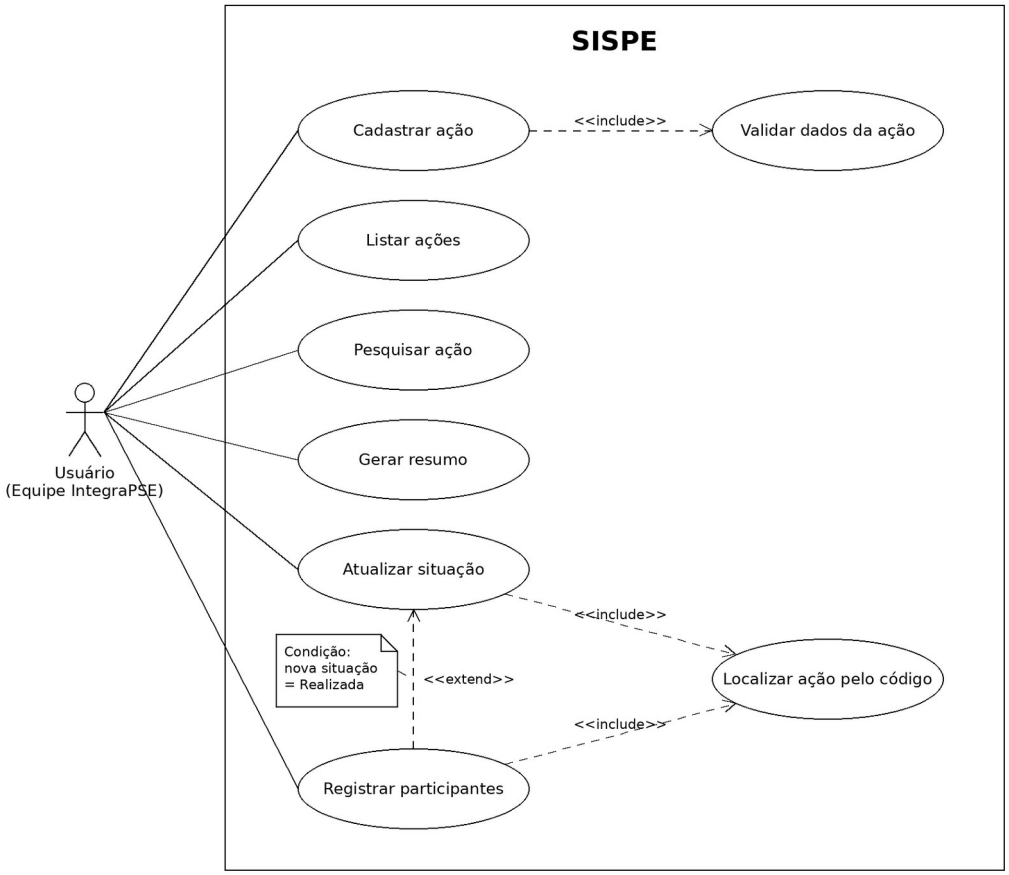
3.2 Diagrama de casos de uso

O diagrama segue o padrão trabalhado na disciplina de Engenharia de Software: fronteira do sistema com o nome no topo, ator fora da fronteira, casos de uso nomeados com verbo e substantivo, associação simples entre ator e caso de uso, e relacionamentos de inclusão e extensão em linha tracejada com seta aberta.

O único ator é o usuário, a equipe intersetorial IntegraPSE (gestor escolar ou profissional de saúde). Ele se associa aos seis casos de uso que correspondem às opções do menu. Os casos incluídos representam comportamentos obrigatórios e reaproveitados: “Cadastrar ação” sempre inclui “Validar dados da ação”, e “Atualizar situação” e “Registrar participantes” sempre incluem “Localizar ação pelo código”. A seta do <<include>> parte do caso base e aponta para o caso incluído.

“Registrar participantes” estende (<<extend>>) “Atualizar situação”: o registro só acontece quando a nova situação escolhida é Realizada, condição indicada na nota ligada ao relacionamento. A seta do <<extend>> parte do caso extensor e aponta para o caso base. Como o usuário também pode corrigir os participantes pela opção 5 do menu, o caso extensor tem ainda uma associação direta com o ator.

Figura 2: Diagrama de casos de uso do SISPE



Fonte: os autores (2026).

3.3 Descrição dos casos de uso

A seguir estão descritos os seis casos de uso principais, um para cada opção do menu. Os dois casos incluídos aparecem resumidos no último quadro.

Quadro 5: UC01 Cadastrar ação

Nome e objetivo	**Cadastrar ação.** Registrar uma nova ação coletiva de saúde mental no vetor de ações.
Ator	Usuário (equipe IntegraPSE)
Pré-condições	• O sistema está em execução e o menu principal está aberto. • Há menos de 50 ações cadastradas.
Fluxo principal	1. O usuário escolhe a opção 1. 2. O usuário informa o código da ação. 3. O usuário informa escola, subtema, data prevista, público-alvo, responsável e quantidade prevista de participantes. 4. Para cada dado informado, o sistema executa o caso incluído Validar dados da ação (UC07). 5. O sistema grava a ação com situação Planejada e exibe “Acao cadastrada com sucesso!” com o código.

Fluxos alternativos	**FA1. Limite atingido:** com 50 ações, o sistema informa o limite e volta ao menu sem pedir dados. **FA2. Código inválido ou duplicado:** o sistema informa o erro, cancela o cadastro e volta ao menu. **FA3. Campo vazio ou texto longo:** mensagem de campo obrigatório ou de texto muito longo; cadastro cancelado. **FA4. Data inválida:** mensagem de data inválida; cadastro cancelado. **FA5. Quantidade inválida** (letras, zero, negativa ou acima de 5000): mensagem de erro; cadastro cancelado.
Pós-condições	A ação fica disponível para listagem, pesquisa, atualização e resumo. Em caso de erro, nenhum dado é salvo.
Requisitos	RF01, RF07, RF09

Fonte: os autores (2026).

Quadro 6: UC02 Listar ações

Nome e objetivo	**Listar ações.** Exibir todas as ações cadastradas em uma tabela.
Ator	Usuário (equipe IntegraPSE)
Pré-condições	• O menu principal está aberto.
Fluxo principal	1. O usuário escolhe a opção 2. 2. O sistema exibe uma tabela com código, escola, subtema, data, público, responsável, previstos, efetivos e situação. 3. O sistema mostra o total de ações ocupadas no vetor.
Fluxos alternativos	**FA1. Nenhuma ação cadastrada:** o sistema exibe “Nenhuma acao cadastrada.”.
Pós-condições	Nenhum dado é alterado.
Requisitos	RF02

Fonte: os autores (2026).

Quadro 7: UC03 Pesquisar ação

Nome e objetivo	**Pesquisar ação.** Localizar ações por código, escola ou subtema.
Ator	Usuário (equipe IntegraPSE)
Pré-condições	• Existe pelo menos uma ação cadastrada.
Fluxo principal	1. O usuário escolhe a opção 3. 2. O sistema pergunta o tipo de pesquisa: 1 código, 2 escola ou 3 subtema. 3. O usuário informa o código ou parte do nome. 4. O sistema mostra a ficha completa de cada ação encontrada e a quantidade encontrada.
Fluxos alternativos	**FA1. Nenhuma ação cadastrada:** mensagem e volta ao menu. **FA2. Tipo de pesquisa inválido:** “Erro: tipo de pesquisa invalido”. **FA3. Nada encontrado:** mensagem de nenhuma ação encontrada.
Pós-condições	Nenhum dado é alterado.
Requisitos	RF03, RF07

Fonte: os autores (2026).

Quadro 8: UC04 Atualizar situação

Nome e objetivo	**Atualizar situação.** Alterar a situação de uma ação para planejada, realizada ou cancelada.
Ator	Usuário (equipe IntegraPSE)
Pré-condições	• Existe pelo menos uma ação cadastrada.
Fluxo principal	1. O usuário escolhe a opção 4 e informa o código. 2. O sistema executa o caso incluído Localizar ação pelo código (UC08). 3. O sistema mostra a ficha da ação e as situações possíveis. 4. O usuário escolhe a nova situação. 5. Ponto de extensão: se a nova situação for Realizada, o sistema executa Registrar participantes (UC05, <<extend>>). 6. O sistema grava a nova situação e confirma a alteração.
Fluxos alternativos	**FA1. Código inexistente:** mensagem de ação não encontrada. **FA2. Situação fora de 1 a 3:** mensagem de situação inválida; nada é alterado. **FA3. Mesma situação atual:** aviso de que nada foi alterado. **FA4. Dados de participação inválidos:** a ação mantém a situação anterior. **FA5. Ação realizada passa para planejada ou cancelada:** os participantes e o estresse são zerados e o sistema avisa o usuário.
Pós-condições	A situação da ação é atualizada e passa a ser considerada no resumo.
Requisitos	RF04, RF07

Fonte: os autores (2026).

Quadro 9: UC05 Registrar participantes

Nome e objetivo	**Registrar participantes.** Registrar ou corrigir a quantidade efetiva de participantes e o estresse médio de uma ação realizada.
Ator	Usuário (equipe IntegraPSE)
Pré-condições	• A ação existe e está com a situação Realizada (ou está sendo marcada como realizada no UC04).
Fluxo principal	1. O usuário escolhe a opção 5 e informa o código. 2. O sistema executa o caso incluído Localizar ação pelo código (UC08). 3. O sistema mostra os participantes previstos, os registrados e o estresse atual. 4. O usuário informa a quantidade efetiva (1 a 5000) e o estresse médio relatado pelo grupo (0 a 10). 5. O sistema grava os valores e confirma o registro.
Fluxos alternativos	**FA1. Código inexistente:** mensagem de ação não encontrada. **FA2. Ação não realizada:** o sistema informa a situação atual e orienta a usar a opção 4. **FA3. Valor inválido:** mensagem de erro; os valores anteriores são mantidos.
Pós-condições	Os participantes da ação são usados no resumo geral e nos resumos por grupo.
Requisitos	RF04, RF08, RF07

Fonte: os autores (2026).

Quadro 10: UC06 Gerar resumo

Nome e objetivo	**Gerar resumo.** Apresentar indicadores agregados das ações, sem dados individuais.
Ator	Usuário (equipe IntegraPSE)

Pré-condições	• Existe pelo menos uma ação cadastrada.
Fluxo principal	1. O usuário escolhe a opção 6. 2. O sistema percorre o vetor e conta as ações planejadas, realizadas e canceladas. 3. O sistema soma os participantes previstos e efetivos e calcula o percentual de participação e o estresse médio. 4. O sistema exibe o resumo geral, o resumo por subtema e o resumo por escola.
Fluxos alternativos	**FA1. Nenhuma ação cadastrada:** “Nenhuma acao cadastrada.”. **FA2. Nenhuma ação realizada:** percentual e estresse aparecem como “sem acoes realizadas”, evitando divisão por zero.
Pós-condições	Nenhum dado é alterado.
Requisitos	RF05, RF06

Fonte: os autores (2026).

Quadro 11: Casos de uso incluídos

Caso incluído	Incluído por	O que faz	Se der erro
UC07 Validar dados da ação	UC01	Confere código (maior que zero e sem repetir), campos obrigatórios com até 49 caracteres, data DD/MM/AAAA existente e quantidade de 1 a 5000.	Mostra o motivo e o cadastro é cancelado sem salvar.
UC08 Localizar ação pelo código	UC04, UC05	Lê o código e percorre o vetor até achar a ação (`localizarAcao` e `buscarPorCodigo`).	Mostra “nenhuma acao encontrada” e nada é alterado.

Fonte: os autores (2026).

3.4 Documentação dos testes

Foram realizados testes funcionais (caixa-preta), com entradas válidas e inválidas e valores limite (negativo, estresse acima de 10 e o 51º cadastro). Cada caso foi executado no programa compilado e a saída foi comparada com o resultado esperado. Ambiente: Ubuntu 22.04 e GCC 11.4.0, em 10/11/2026. Foram executados 19 casos de teste e 19 foram aprovados.

Quadro 12: Casos de teste executados

ID	Funcionalidade	Cenário	Dados utilizados	Resultado esperado	Resultado obtido (saída do programa)	Situação
CT01	Cadastrar ação	Cadastro com todos os campos válidos	Cod. 1; EM Monteiro Lobato; Bullying; 15/09/2026; 7o ano; Ana Souza; 30	Ação salva com situação Planejada e contador do menu passa para 1	Acao cadastrada com sucesso! Codigo: 1 | Situacao: Planejada; Acoes cadastradas: 1 de 50	Aprovado
CT02	Cadastrar ação	Código duplicado	Cadastro do código 1 e nova tentativa com o código 1	Mensagem de código existente; total continua 1	Erro: ja existe uma acao com o codigo 1.; Cadastro cancelado. A saída não contém: “Acoes cadastradas: 2 de 50”	Aprovado
CT03	Cadastrar ação	Código com letras	Código: abc	Mensagem de código inválido; nada salvo	Erro: o codigo deve ser um numero inteiro maior que zero.; Cadastro cancelado	Aprovado
CT04	Cadastrar ação	Campo obrigatório vazio (escola só com espaços)	Código 7; Escola: ' '	Mensagem de campo obrigatório; nada salvo	Erro: este campo e obrigatorio e nao pode ficar vazio.; Cadastro cancelado. A saída não contém: “cadastrada com sucesso”	Aprovado
CT05	Cadastrar ação	Data inexistente	Data: 31/02/2026	Mensagem de data inválida; nada salvo	Erro: data invalida.. A saída não contém: “cadastrada com sucesso”	Aprovado

CT06	Cadastrar ação	Quantidade prevista negativa	Quantidade: -10	Mensagem de quantidade inválida; nada salvo	Erro: a quantidade prevista deve ser um numero inteiro de 1 a 5000.. A saída não contém: “cadastrada com sucesso”	Aprovado
CT07	Cadastrar ação	Limite de armazenamento atingido	50 cadastros válidos (códigos 1 a 50) e tentativa do 51º	Mensagem de limite; 51º não é pedido nem salvo	Acoes cadastradas: 50 de 50; Erro: limite de 50 acoes atingido. Nao e possivel cadastrar novas acoes.. A saída não contém: “Acoes cadastradas: 51”	Aprovado
CT08	Listar ações	Lista com ações cadastradas	Ações 1 e 2 cadastradas	Tabela com as duas ações e total 2	1 EM Monteiro Lobato Bullying; 2 CE Vital Brazil Uso de telas; Total: 2 acao(oes) cadastrada(s) de 50 possiveis.	Aprovado
CT09	Pesquisar ação	Pesquisa por código existente	Código 2	Ficha completa da ação 2	Codigo .....: 2; Escola .....: CE Vital Brazil	Aprovado
CT10	Pesquisar ação	Pesquisa por código inexistente	Código 99	Mensagem de nenhuma ação encontrada	Erro: nenhuma acao encontrada com o codigo informado.	Aprovado
CT11	Pesquisar ação	Pesquisa por parte do nome da escola, em minúsculas	Escola: monteiro	Encontra as 2 ações da EM Monteiro Lobato	2 acao(oes) encontrada(s).	Aprovado
CT12	Atualizar situação	Marcar como realizada com dados válidos (vírgula decimal)	Código 1; situação 2; 27 participantes; estresse 6,5	Situação Realizada; ficha mostra 27 e 6.5	Situacao da acao 1 atualizada para: Realizada; Participantes efetivos ..: 27; Estresse medio do grupo .: 6.5	Aprovado
CT13	Atualizar situação	Código inexistente	Código 99	Mensagem de ação não encontrada	Erro: nenhuma acao encontrada com o codigo informado.	Aprovado
CT14	Atualizar situação	Realizada com estresse fora da escala	Código 1; situação 2; 27 participantes; estresse 11	Erro; ação continua Planejada	Erro: o estresse medio deve ser um numero de 0 a 10.; - Planejada	Aprovado
CT15	Atualizar situação	Realizada passa para cancelada	Código 1 realizada (27; 6,5) e depois situação 3	Aviso de remoção dos participantes; resumo sem participantes	Aviso: os participantes registrados foram removidos; Situacao da acao 1 atualizada para: Cancelada; Participantes efetivos (realizadas) .: 0	Aprovado
CT16	Registrar participantes	Correção dos participantes de ação realizada	Código 1 realizada com 27; novo registro 30 e estresse 5	Confirmação com 30 participantes	Participantes da acao 1 registrados com sucesso: 30 participante(s).	Aprovado
CT17	Registrar participantes	Ação ainda planejada	Código 1 (planejada)	Erro orientando a usar a opção 4	So e possivel registrar participantes de acoes realizadas.; Use a opcao 4	Aprovado
CT18	Gerar resumo	Resumo com ações em todas as situações	Ação 1 realizada (30 prev., 27 part., estresse 6); ação 2 realizada (40 prev., 30 part., estresse 7); ação 3 planejada (25); ação 4 cancelada (20)	1 planejada, 2 realizadas, 1 cancelada; 115 previstos; 57 efetivos; 81,4%; estresse 6,5; Bullying e bullying num só grupo	Planejadas .....: 1; Realizadas .....: 2; Canceladas .....: 1; Participantes previstos (todas) .....: 115; Participantes efetivos (realizadas) .: 57; Percentual de participacao .....: 81.4%; Estresse medio relatado (0 a 10) ....: 6.5; Bullying 2 0 1 1 27 6.0	Aprovado
CT19	Menu	Opção inexistente	Opção 9	Mensagem de opção inválida e menu de novo	Erro: opcao invalida! Digite um numero de 0 a 6.	Aprovado

Fonte: os autores (2026).

Situações exigidas: entradas inválidas (CT03 a CT06, CT14 e CT19), código duplicado (CT02), registros inexistentes (CT10 e CT13) e limite de armazenamento (CT07). O CT18 conferiu os cálculos do resumo: 57 participantes sobre 70 previstos nas ações realizadas (81,4%) e estresse médio (6,0 + 7,0) / 2 = 6,5.

Quadro 13: Rastreabilidade entre requisitos, funções, casos de uso e testes

Requisito	Função	Caso de uso	Testes
RF01	`cadastrarAcao`, `lerDadosAcao`	UC01, UC07	CT01 a CT07

RF02	`listarAcoes`	UC02	CT08
RF03	`pesquisarAcoes`	UC03	CT09 a CT11
RF04	`atualizarSituacao`, `localizarAcao`	UC04, UC08	CT12 a CT15
RF05	`gerarResumo`	UC06	CT18
RF06	`agruparAcoes`, `mostrarGrupos`	UC06	CT18
RF07	`lerInteiro`, `lerTexto`, `dataValida`	UC07	CT02 a CT06, CT14, CT19
RF08	`registrarParticipantes`, `localizarAcao`	UC05, UC08	CT16, CT17
RF09	`cadastrarAcao` (limite)	UC01	CT07

Fonte: os autores (2026).

4 MANUAL DO USUÁRIO

4.1 Apresentação e objetivo do sistema

O SISPE é um programa de terminal que apoia a equipe intersetorial do Programa Saúde na Escola no planejamento e no acompanhamento de ações coletivas de promoção da saúde mental, como rodas de conversa sobre bullying, uso de telas ou ansiedade. Com ele, o usuário cadastra as ações planejadas em cada escola, acompanha se foram realizadas ou canceladas, registra quantas pessoas participaram e o estresse médio relatado pelo grupo, e consulta um resumo da cobertura por subtema e por escola.

O sistema é uma ferramenta de organização. Ele não substitui o profissional de saúde, não faz diagnóstico, triagem ou recomendação de tratamento e não deve receber nome, prontuário ou qualquer dado individual de estudante. Utilize apenas dados fictícios ou informações coletivas da ação.

4.2 Requisitos para utilização

- Computador com Windows, Linux ou macOS.
- Um compilador da linguagem C: GCC (Linux), MinGW ou Dev-C++ (Windows) ou o compilador do Xcode (macOS).
- Os cinco arquivos do código-fonte na mesma pasta: `main.c`, `acoes.c`, `acoes.h`, `entrada.c` e `entrada.h`.
- Não é necessário internet, banco de dados ou instalação de bibliotecas extras.

4.3 Instruções de compilação

Linux ou macOS (terminal): abra o terminal na pasta do código-fonte e execute:

```
gcc main.c acoes.c entrada.c -o sispe
```

Windows com MinGW (Prompt de Comando): na pasta do código-fonte, execute:

```
gcc main.c acoes.c entrada.c -o sispe.exe
```

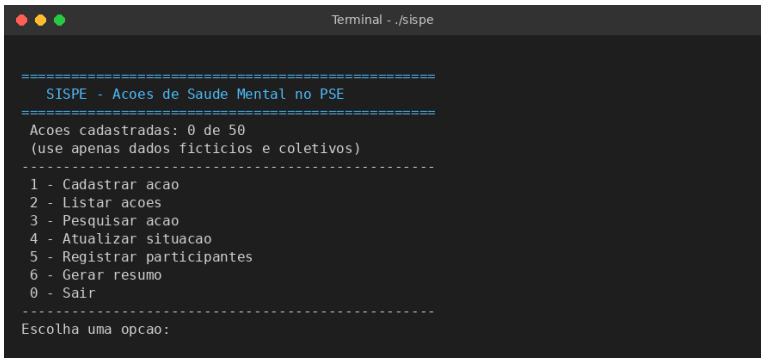
Windows com Dev-C++: como o programa tem mais de um arquivo, é preciso criar um projeto:

1. Menu Arquivo > Novo > Projeto, escolha “Console Application”, marque “Projeto C” e salve o projeto na pasta do código.
2. Remova do projeto o arquivo `main.c` criado automaticamente pelo Dev-C++ (clique com o botão direito > Remover arquivo), sem apagar o `main.c` do SISPE.
3. Menu Projeto > Adicionar ao projeto e selecione os cinco arquivos do SISPE.
4. Pressione F9 para compilar ou F11 para compilar e executar.

4.4 Instruções de execução

No Linux ou macOS, execute `./sispe`. No Windows, execute `sispe.exe` no Prompt de Comando ou pressione F10 no Dev-C++. O menu principal é exibido e o sistema funciona assim: digite o número da opção e pressione ENTER; depois de cada operação, pressione ENTER para voltar ao menu. Os dados ficam guardados enquanto o programa estiver aberto e são apagados ao escolher a opção 0.

Figura 3: Menu principal do SISPE (execução real no terminal)



Fonte: os autores (2026).

4.5 Opções do menu

Quadro 14: Opções do menu principal

Opção	Função	Dados pedidos e regras
1	Cadastrar ação	Código (inteiro maior que zero, sem repetir), escola, subtema, data prevista (DD/MM/AAAA), público-alvo, responsável e quantidade prevista (1 a 5000). Textos com até 49 caracteres. A ação começa como Planejada.
2	Listar ações	Nenhum. Mostra a tabela com todas as ações. Nomes longos aparecem abreviados na tabela, mas ficam completos na pesquisa.
3	Pesquisar ação	Tipo da pesquisa (1 código, 2 escola, 3 subtema) e o valor. Para escola e subtema, basta parte do nome, com letras maiúsculas ou minúsculas.

4	Atualizar situação	Código e nova situação (1 Planejada, 2 Realizada, 3 Cancelada). Ao escolher Realizada, informe os participantes (1 a 5000) e o estresse médio do grupo (0 a 10).
5	Registrar participantes	Código de uma ação realizada, nova quantidade de participantes e estresse médio. Serve para corrigir a contagem depois do evento.
6	Gerar resumo	Nenhum. Mostra totais por situação, participantes, percentual de participação, estresse médio e as tabelas por subtema e por escola.
0	Sair	Encerra o programa. Os dados cadastrados são descartados.

Fonte: os autores (2026).

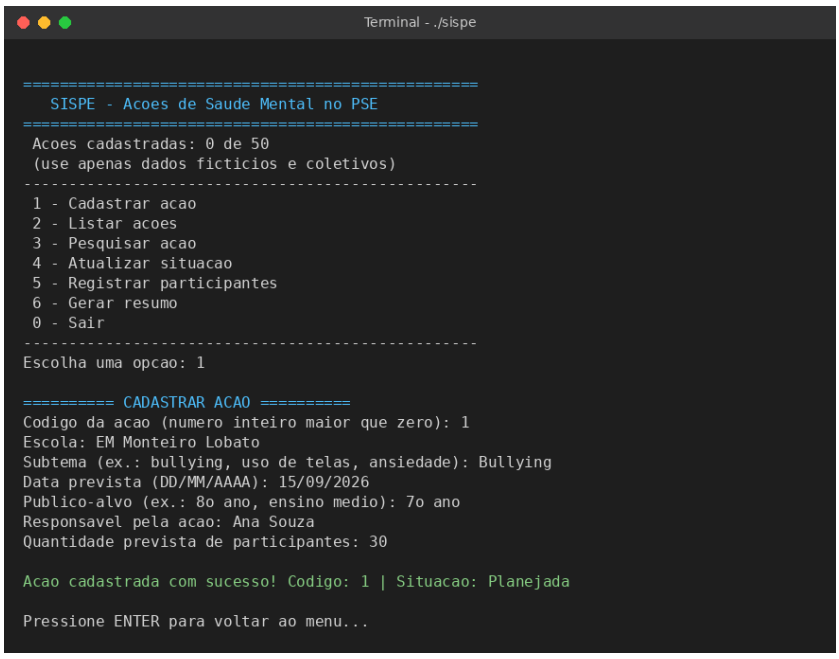
Sempre que um dado é recusado, o sistema explica o motivo e cancela a operação sem alterar nada. Basta escolher a opção novamente e digitar o dado correto.

4.6 Exemplo prático de utilização

O exemplo a seguir simula uma equipe que acompanha quatro ações fictícias em três escolas. Todas as figuras são capturas da execução real do programa no terminal.

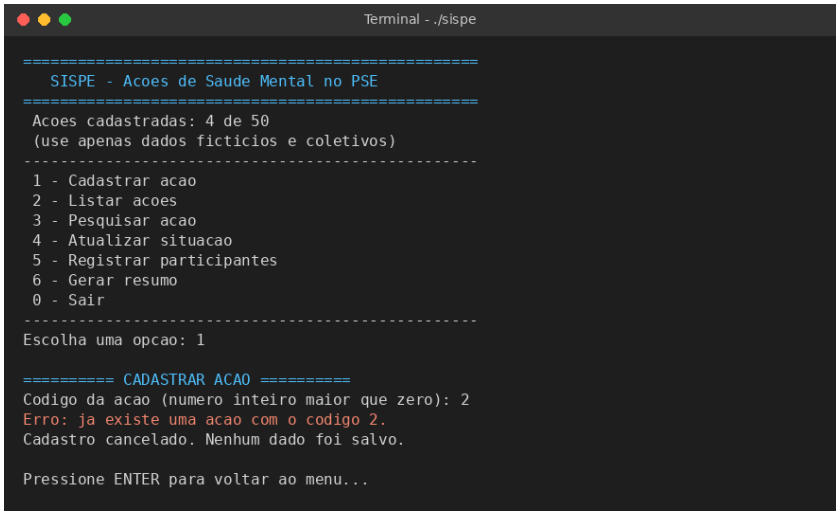
Passo 1: Cadastrar. Na opção 1, foram cadastradas as ações 1 (EM Monteiro Lobato, Bullying, 30 previstos), 2 (CE Vital Brazil, Uso de telas, 40), 3 (EM Monteiro Lobato, Ansiedade, 25) e 4 (EE Castro Alves, bullying, 20). Ao repetir o código 2, o cadastro é recusado.

Figura 4: Cadastro de uma ação com sucesso (execução real no terminal)



Fonte: os autores (2026).

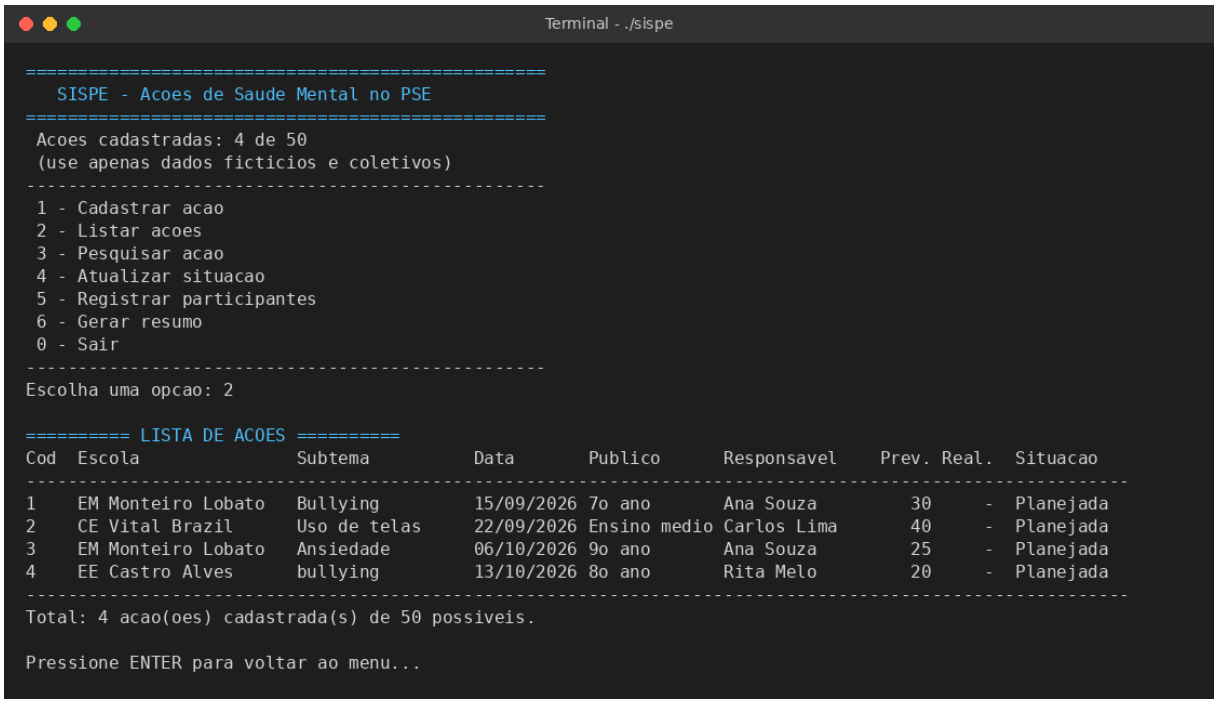
Figura 5: Tentativa de cadastro com código já existente (execução real no terminal)



Fonte: os autores (2026).

Passo 2: Listar e pesquisar. A opção 2 mostra as quatro ações. Na opção 3, tipo 2, basta digitar “monteiro” para encontrar as duas ações da EM Monteiro Lobato.

Figura 6: Listagem das ações cadastradas (execução real no terminal)



Fonte: os autores (2026).

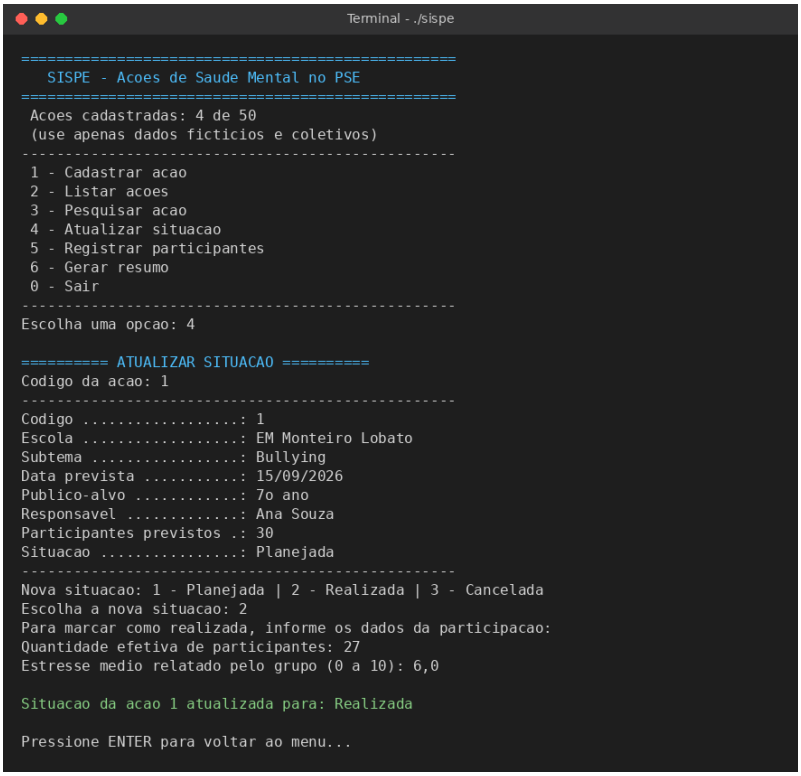
Figura 7: Pesquisa por parte do nome da escola (execução real no terminal)



Fonte: os autores (2026).

Passo 3: Atualizar a situação. A ação 1 foi marcada como Realizada (27 participantes, estresse 6,0), a ação 2 também (30 participantes, estresse 7) e a ação 4 foi cancelada.

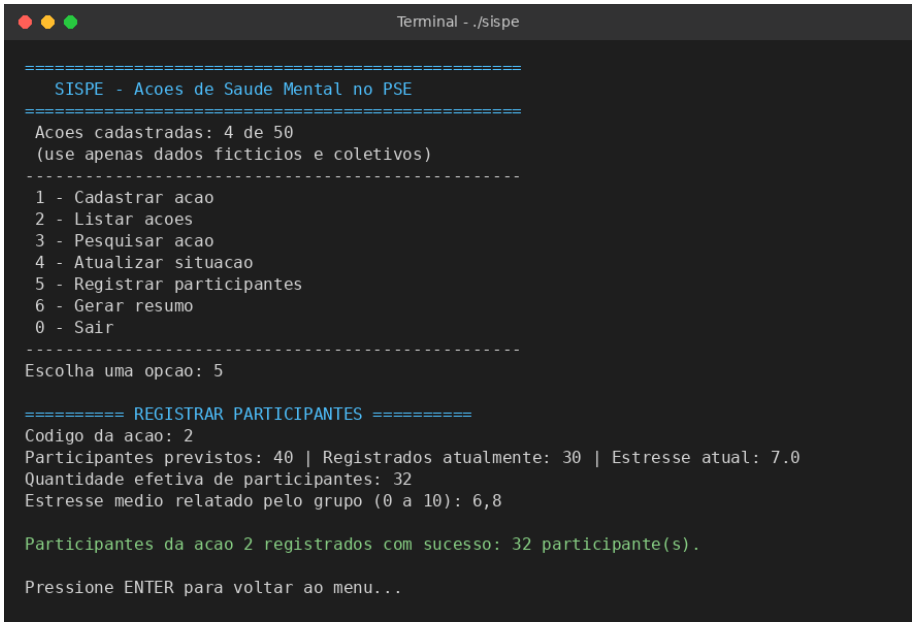
Figura 8: Atualização da situação para Realizada (execução real no terminal)



Fonte: os autores (2026).

Passo 4: Registrar participantes. Depois do evento, a contagem da ação 2 foi corrigida para 32 participantes, com estresse 6,8. Em uma ação ainda planejada, o registro é recusado.

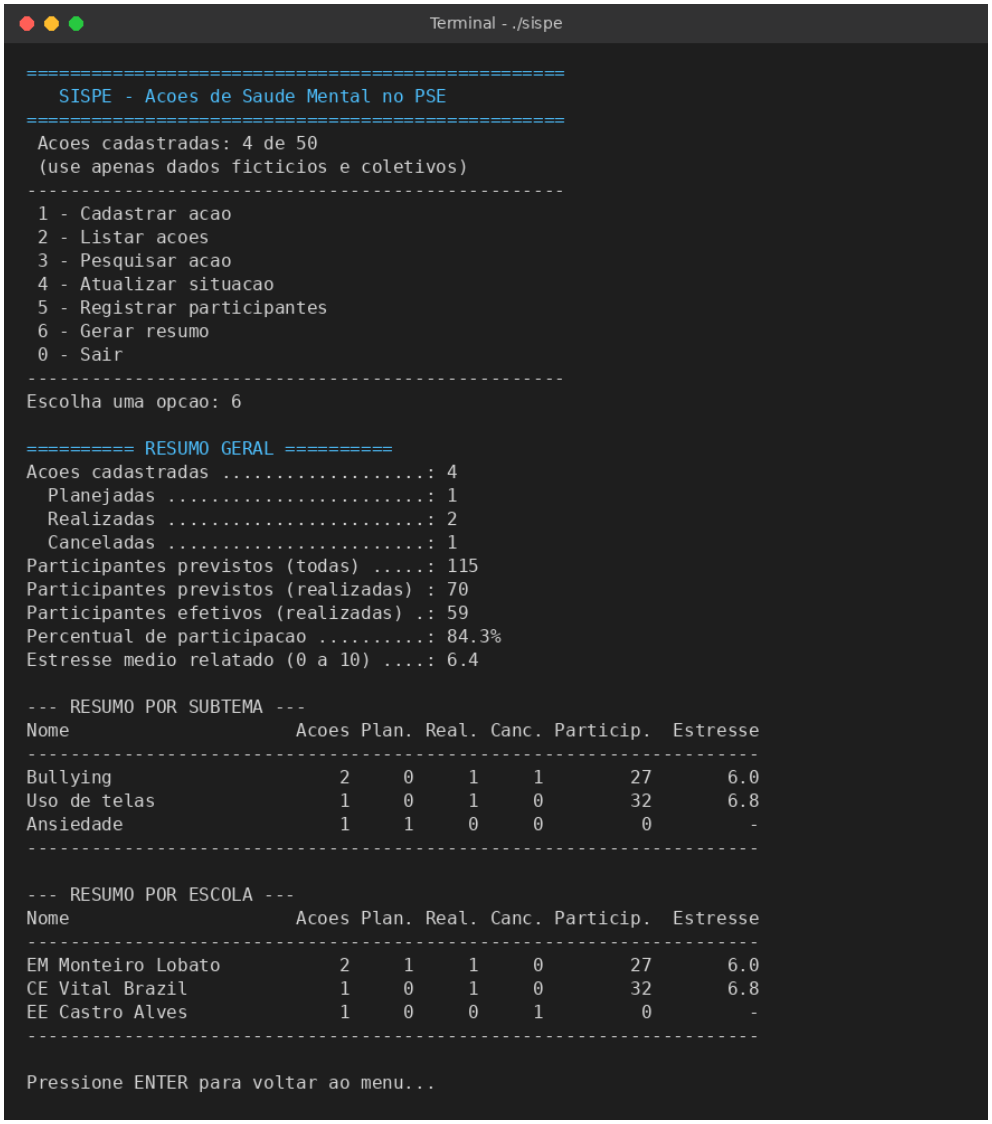
Figura 9: Registro (correção) dos participantes de uma ação realizada (execução real no terminal)



Fonte: os autores (2026).

Passo 5: Gerar o resumo. A opção 6 mostra 1 ação planejada, 2 realizadas e 1 cancelada, com 59 participantes sobre 70 previstos (84,3%). O resumo por subtema junta “Bullying” e “bullying” e mostra que Ansiedade ainda não teve ação realizada.

Figura 10: Resumo geral, por subtema e por escola (execução real no terminal)



Fonte: os autores (2026).

4.7 Mensagens de erro mais comuns

Quadro 15: Mensagens de erro e como resolvê-las

Mensagem	Causa	O que fazer
Erro: opcao invalida! Digite um numero de 0 a 6.	Número fora do menu ou letras.	Digite um número de 0 a 6.
Erro: ja existe uma acao com o codigo X.	Código repetido.	Use outro código; consulte a lista (opção 2).
Erro: este campo e obrigatorio e nao pode ficar vazio.	Campo em branco.	Preencha o campo.
Erro: texto muito longo (maximo de 49 caracteres).	Texto acima do limite.	Abrevie o texto.
Erro: data invalida.	Formato errado ou data	Use DD/MM/AAAA, por exemplo

	inexistente.	15/09/2026.
Erro: a quantidade ... deve ser um numero inteiro de 1 a 5000.	Quantidade zero, negativa, com letras ou acima do limite.	Digite um número inteiro no intervalo.
Erro: o estresse medio deve ser um numero de 0 a 10.	Valor fora da escala.	Digite um valor de 0 a 10 (ex.: 6,5).
Erro: nenhuma acao encontrada com o codigo informado.	Código inexistente.	Confira o código na lista.
So e possivel registrar participantes de acoes realizadas.	A ação está planejada ou cancelada.	Marque como realizada na opção 4.
Erro: limite de 50 acoes atingido.	O vetor está cheio.	Não é possível cadastrar mais ações nesta execução.

Fonte: os autores (2026).

5 CONSIDERAÇÕES FINAIS

5.1 Resultados alcançados

O SISPE foi implementado em linguagem C e atende às seis funcionalidades mínimas da AEP: cadastrar, listar, pesquisar, atualizar a situação, registrar participantes e gerar o resumo. O programa compila sem erros nem avisos e foi aprovado nos 19 casos de teste documentados, que incluem entradas inválidas, códigos duplicados, registros inexistentes e o limite de armazenamento. A implementação seguiu os fluxogramas e pseudocódigos da 1ª entrega, com as mudanças justificadas na revisão dos requisitos.

5.2 Atendimento aos objetivos propostos

Quadro 16: Atendimento aos objetivos da 1ª entrega

Objetivo da 1ª entrega	**Como foi atendido**
Geral: apoiar o planejamento, o registro e o acompanhamento de ações de saúde mental, visualizando cobertura e continuidade sem dados clínicos individuais.	O sistema registra ações coletivas, acompanha sua situação e mostra a cobertura por subtema e por escola. Nenhum campo armazena dados de estudantes.
Específico 1: cadastro estruturado com escola, subtema e público-alvo.	Opção 1, com estrutura `Acao` e validação de todos os campos.
Específico 2: busca e atualização de status.	Opções 3, 4 e 5: pesquisa por código, escola ou subtema; atualização de situação e registro de participantes.
Específico 3: indicadores agregados por subtema e por escola.	Opção 6: totais por situação, percentual de participação, estresse médio e tabelas por grupo.

Fonte: os autores (2026).

5.3 Limitações identificadas

- Os dados ficam apenas em memória e são perdidos quando o programa é encerrado.
- O vetor comporta no máximo 50 ações por execução.
- Não é possível editar os dados de cadastro (escola, data etc.) nem excluir uma ação; apenas a situação e os participantes podem ser alterados.

- A comparação de textos ignora maiúsculas apenas em letras sem acento: “São” e “SÃO” são tratados como nomes diferentes. Nomes com acentos também podem desalinhar as colunas da tabela no terminal.
- Na listagem, nomes longos aparecem abreviados para caber na tabela.
- O estresse médio é um valor informado pela equipe a partir do relato do grupo; o sistema não aplica nenhum instrumento validado de medida.
- Os testes foram executados em Linux. A compilação no Windows deve ser confirmada no computador do laboratório.

5.4 Dificuldades encontradas

- **Leitura de dados:** o `scanf` entra em laço infinito quando o usuário digita letras em um campo numérico. A solução foi ler a linha inteira com `fgets` e converter com `sscanf`.
- **Agrupamento por subtema e por escola:** sem estruturas avançadas, foi preciso criar um vetor de `struct Grupo`, procurar cada nome antes de somar e comparar os textos sem diferenciar maiúsculas. A mesma função também calcula o resumo geral, o que evitou repetir a contagem por situação.
- **Consistência entre situações:** foi necessário definir regras para quando uma ação realizada volta a planejada ou é cancelada, para que os participantes não continuassem no resumo.
- **Decisões que a 1ª entrega não detalhava:** a escala do estresse médio, a forma de gerar o código e a base de cálculo do percentual precisaram ser definidas e justificadas durante a implementação.

5.5 Aprendizados obtidos

O projeto mostrou na prática como o planejamento orienta a programação: os pseudocódigos e fluxogramas viraram funções quase diretamente, e as decisões que não estavam claras no planejamento apareceram como dúvidas durante a codificação. Também consolidou o uso de `struct`, vetores, funções, condicionais e laços, a divisão do código em arquivos `.c` e `.h`, e a importância de validar toda entrada do usuário. Do ponto de vista social, reforçou que um sistema simples pode tornar visível o que já foi feito em cada escola, sem ultrapassar os limites éticos de lidar com dados de estudantes.

5.6 Possibilidades de melhorias futuras

- Gravar e ler as ações em arquivo binário (`.dat`), conteúdo da disciplina, para que os dados não se percam ao fechar o programa.
- Permitir editar os dados de uma ação e fazer a exclusão lógica, com um campo `ativo`.
- Filtrar o resumo por período (mês ou bimestre) e por público-alvo.
- Exportar o resumo para um arquivo de texto, facilitando o envio às equipes de saúde e educação.
- Usar uma lista de subtemas pré-definida para evitar nomes escritos de formas diferentes.

6 APRESENTAÇÃO EM VÍDEO

A apresentação em vídeo tem duração máxima de 7 minutos, conta com a participação dos dois integrantes e mostra o programa em execução. O roteiro completo está no arquivo “Roteiro_do_Video”.

Link do vídeo: [INSERIR O LINK DO VÍDEO COM ACESSO LIBERADO ANTES DE ENVIAR]

Repositório GitHub: https://github.com/ashmuchluv/SISPE

REFERÊNCIAS

BRASIL. Decreto nº 6.286, de 5 de dezembro de 2007. Institui o Programa Saúde na Escola - PSE e dá outras providências. **Diário Oficial da União**: Brasília, DF, 6 dez. 2007.

BRASIL. Lei nº 14.819, de 16 de janeiro de 2024. Institui a Política Nacional de Atenção Psicossocial a Comunidades Escolares. **Diário Oficial da União**: Brasília, DF, 17 jan. 2024.

BRASIL. Ministério da Saúde. **O que é o Programa Saúde na Escola (PSE)?** Brasília, DF, 2025a. Disponível no portal do Ministério da Saúde. Acesso em: 24 jul. 2026.

BRASIL. Ministério da Saúde. **Quais são as ações do PSE?** Brasília, DF, 2025b. Disponível no portal do Ministério da Saúde. Acesso em: 24 jul. 2026.

SOUSA, Marta Caires de; ESPERIDIÃO, Monique Azevedo; MEDINA, Maria Guadalupe. A intersetorialidade no Programa Saúde na Escola: avaliação do processo político-gerencial e das práticas de trabalho. **Ciência & Saúde Coletiva**, Rio de Janeiro, v. 22, n. 6, p. 1781-1790, 2017.